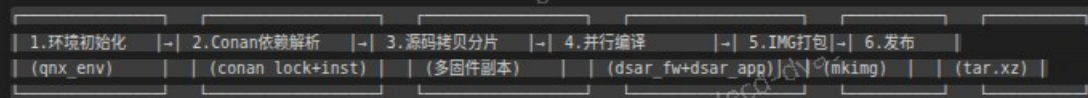


现在我已经掌握了完整的编译链条，下面给出系统性的分析。

dsar-hq 完整编译链条分析

一、总体编译流程（6个阶段）



阶段1：环境初始化

- 加载 QNX SDP 7.1 编译环境 (qnxmdp-env.sh)
- 设置交叉编译工具链: aarch64-unknown-nto-qnx7.1.0-*
- 配置 sccache 分布式编译缓存 (命中率 426/431 = 98.8%)
- 拷贝 QNX cmake toolchain 文件 (qnx_aarch64le_toolchain_iosock.cmake)

阶段2：Conan 依赖解析

- 从 conan_config.json 解析 platformDesc，找到该车型需要编译的固件列表
- 从 conan_version_dsar_plat*.py 读取平台仓版本号
- conan lock create - 锁定依赖版本图
- conan install - 生成 conanbuildinfo.cmake 和 conanbuildinfo.mak

阶段3：源码拷贝分片

根据 multi_pkg=dsar,dsar-app 和车型 fawhq_e009，创建 3 份源码副本。

副本名称	固件ID	作用	编译参数
dsar_fawhq_e009-sailsw3-soc1_sail_app2	sailsw3	FAW红旗 MCU固件	arch=sa8650 oem=faw sdk_version=sdk12
dsar_sailsw1-sailsw1-soc1_sail_app1	sailsw1	QCOM通用 MCU固件	core=sailsw1 arch=sa8650 oem=qcom sdk_version=sdk12
dsar_sailhyp-sailhyp-sail_hyp	sailhyp	Hypervisor	预编译，无需 make

关键发现：sailsw1 副本是在 sailsw3 副本基础上 copy 的 (cp -r dsar_fawhq_e009.* dsar_sailsw1.*)，共用一个代码基，通过不同 make 参数区分。

阶段4：并行编译（核心）

两条编译流水线并行执行：



二、dsar_fw（MCU固件）编译链条详解

构建系统：Makefile（非 CMake）

入口：src/dsar_fw/build/main_sa8xx.mk

```
# 编译目标链（从构建日志提取）：
target:app_core -> 核心应用模块
target:ctrl -> 车辆控制
format:dsar_fw -> AUTOSAR BSW (Git Submodule)
```

构建系统：Makefile（非 CMake）

入口：src/dsar_fw/build/main_sa8xx.mk

```
# 编译目标链（从构建日志提取）：
target:app_core -> 核心应用模块
target:ctrl -> 车辆控制
format:dsar_fw -> AUTOSAR BSW (Git Submodule)
```

构建系统：Makefile（非 CMake）

入口：src/dsar_fw/build/main_sa8xx.mk

```
# 编译目标链（从构建日志提取）：
target:app_core -> 核心应用模块
target:ctrl -> 车辆控制
target:dsar_bsw -> AUTOSAR BSW (Git Submodule)
target:product -> 产品配置
```

```
target:fusa          -> 功能安全
target:control       -> 控制算法 (Git Submodule)
target:ad_product/   -> AD产品配置
target:service       -> 服务层
target:degrade_table -> 降级表
target:Common        -> 公共代码
target:libs          -> 第三方库
target:Math          -> 数学库
target:crc           -> CRC校验
target:cdd           -> CDD (Complex Device Driver)
target:imu           -> IMU驱动
target:imu_v1        -> IMU v1驱动
```

MCU固件模块依赖关系

```
dsar_fw 顶层固件 (BIN/ELF)
|
|-- dsar-bsw (Git Submodule) - AUTOSAR基础软件
|   |-- MCAL 驱动 (CAN/SPI/I2C等)
|   |-- OS (Autosar OS)
|   |-- 通信栈 (CAN TP/DCM/DEM等)
|
|-- dsar-plat (Conan包) - MCU平台公共代码
|   |-- acu_device/      (传感器/规划/诊断设备抽象)
|   |-- rtmon/           (运行时监控)
|   |-- sync/            (同步)
|   |-- CalibPlat/       (标定平台)
|   |-- pm_sync/         (电源模式同步)
|   |-- xwire/           (X-Wire通信)
|   |-- proxy_comm/      (代理通信)
|
|-- app_core/ - MCU核心应用 (dsar-hq适配仓自有代码)
|   |-- bsw_manager_v2/  - BSW管理器
|   |-- can_com_platform/ - CAN通信平台
|   |-- can_gateway/     - CAN网关
|   |-- diag/            - 诊断 (DiagSleepStorage/DmmCalibMode/NvMSync)
|   |-- dmm/             - 诊断内存管理
|   |-- dsar_plat_init/  - 平台初始化
|   |-- fusa/            - 功能安全 (degrade_table/sysdiag_postproc)
|   |-- osal/            - OS抽象层
|   |-- pm_sync/         - 电源模式同步
|   |-- rtmon/           - 运行时监控
|   |-- secoc/           - 安全板载通信
|   |-- version_app/     - 版本管理
|
|-- ctrl/ - 控制逻辑 (Git Submodule: control)
|   |-- control/         - 控制算法
|   |-- support/math/    - 数学库
|
|-- proxy_comm/ - 代理通信 (dsar-hq自有代码)
|   |-- blf_recorder/    - BLF录制
|   |-- comm/            - 通信
|   |-- proxy_sensor/    - 传感器代理
|   |-- recorder_common/ - 录制公共
|
|-- xwire/ - X-Wire通信 (dsar-hq自有代码)
|   |-- acu_shell/       - ACU Shell
|   |-- oemdata_ctrl_dji_msg/ - OEM数据控制
|   |-- sim/             - 仿真
|   |-- vehicle_info/    - 车辆信息
|   |-- vehicle_time/    - 车辆时间
|
|-- product/ - 车型差异化配置
|   |-- common_config/sa86xx_config/ - SA8650平台配置
|   |-- ad_product/faw/      - FAW AD产品配置
|   |-- faw/fawhq_e009/     - e009车型配置
|   |-- qcom/sailsw1/       - sailsw1通用配置
```

dsar_fw 产物

产物	说明
*.bin	MCU 固件二进制
*.elf	MCU 固件 ELF (含调试符号)
*.map	链接 Map 文件

三、dsar_app (SOC应用) 编译链条详解

二、dsar_app (SOC应用) 编译链接详解

构建系统: CMake

入口: src/dsar_app/CMakeLists.txt

顶层可执行文件 (entry/)

可执行文件	入口源文件	依赖的 .so
dji_ad_app	dji_ad_app.cpp	libs{product}_app.so + libs{product}_com.so + libuds_v4.so + libs{product}_uds_v6.so
dji_bf_app	dji_bf_app.cpp	libs{product}_com.so + libuds_v4.so + libs{product}_uds_v6.so
dji_doip_service_ipv4	dji_uds_service_v4.cpp	libuds_v4.so
dji_doip_service_ipv6	dji_uds_service_v6.cpp	libs{product}_uds_v6.so

注: 当 CONFIG_APP_GE_PROCESS=false 时, dji_ad_app 和 dji_bf_app 合并为 dji_application

SOC动态库 (.so) 编译链接

每个动态库都是一个用 -whole-archive 将多个静态库打包链接而成:

```
libfawhq_e009_app.so (AD应用动态库)
= --whole-archive {
    vehicle_proxy_fawhq_e009 - 车辆信号代理 (proxy/*.json定义信号)
    adapter_fawhq_e009 - 适配器 (行泊车信号适配)
    param_fawhq_e009 - PAL参数 (自动生成)
    fusa_fawhq_e009 - 功能安全 (平台仓提供)
    product_runnable_fawhq_e009 - 可运行实体
    dsar_plat_config - 平台配置
    plat_ad_app_core - AD应用核心 (平台仓提供)
    plat_ad_fusa_app - 平台AD功能安全
    plat_ad_demo - 平台AD示例
    plat_vehicle_proxy - 平台车辆代理
    plat_common - 平台公共
    plat_ad_tlv - 平台AD TLV
    plat_ad_cndtm - 平台AD条件
    app_ia_plat - IA平台
    qpc - QPC框架 (第三方)
    x_dom_can_rt_gen_lib_app* - CAN信号路由 (自动生成代码)
    x_dom_someip_rt_gen_lib_app* - SOME/IP信号路由 (自动生成代码)
    gateway_hmicom - HMI通信网关
    gateway_mqttcom - MQTT通信网关 (可选)
    adc_protobuf - ADC Protobuf
    mini_dcos - 轻量DCOS
    plat_bf_cdd - 平台BF CDD (来自dsar-plat-bf包)
} --no-whole-archive
+ gateway_znet dji_stream_media_protocol (动态链接)
```

```
libfawhq_e009_com.so (通信动态库)
= --whole-archive {
    autosar_com_fawhq_e009 - AUTOSAR通信适配
    oem_proxy_fawhq_e009 - OEM信号代理
    oem_feature_fawhq_e009 - OEM特性
    bf_oem_feature_fawhq_e009 - BF OEM特性
    x_dom_can_rt_gen_lib_com* - CAN信号路由 (自动生成)
    x_dom_someip_rt_gen_lib_com* - SOME/IP信号路由 (自动生成)
    app_core_libs - 应用核心库
    dcos_dcms - DCOS DCMS
    plat_bf_cdd - 平台BF CDD
} --no-whole-archive
```

```
libuds_v4.so (UDS诊断v4动态库, 多车型共享)
= --whole-archive {
    autosar_uds_${product}_v4 - AUTOSAR UDS诊断适配
    x_dom_can_rt_gen_lib_uds*_v4 - CAN UDS路由 (自动生成)
    x_dom_someip_rt_gen_lib_uds*_v4 - SOME/IP UDS路由 (自动生成)
    dcos_dcms - DCOS DCMS
    plat_bf_cdd - 平台BF CDD
} --no-whole-archive
+ mini_dcos (动态链接)
```

```
libfawhq_e009_uds_v6.so (UDS诊断v6动态库, 按车型)
= --whole-archive {
    autosar_uds_${product}_v6 - AUTOSAR UDS诊断适配
    x_dom_can_rt_gen_lib_uds*_v6 - CAN UDS路由 (自动生成)
    x_dom_someip_rt_gen_lib_uds*_v6 - SOME/IP UDS路由 (自动生成)
    dcos_dcms - DCOS DCMS
    plat_bf_cdd - 平台BF CDD
} --no-whole-archive
+ mini_dcos (动态链接)
```


类似 UDS_V4，但使用 V6 版本的 AUTOSAR 规范和代码

自动生成代码 (x_dom_ 开头)

关键: x_dom_can_rt 和 x_dom_someip_rt 系列库的源文件来自 dsar-plat-bf Conan 包 中的预生成代码:

```
~/conan/data/dsar-plat-bf/2.31.0-beta.6/.../res/src/cdd/x_dom_can_rt/
├── sig_route_soc.cpp      - 信号路由 (SOC侧)
├── msg_route_soc.cpp      - 消息路由 (SOC侧)
├── p2p_parser.cpp         - 点对点解析
└── x_dom_can_rt_soc.cpp   - CAN运行时 (SOC侧)
```

PAL 参数工具 (pal_tool.cmake) 还负责将 JSON 信号描述文件转换为 C++ 代码, 具体流程:

```
proxy/*.json 信号定义
├──
└── pal_tool (来自 dsar-plat-bf 包)
PAL 自动生成的 C++ 参数代码
├──
└── 编译进 vehicle_proxy / param 等库
最终 .so
```

四、Conan 依赖关系图 (完整)

```
dsar-hq (conanfile.py)
├── dji-message/2.2.0-alpha.4@TPV1/release - DJI消息定义 (公共协议)
├── bsp_platform_utils/1.0.0-alpha.10@aep/stable - 平台工具 (ztrace)
├── daf-dins/1.30.0@daf-cicd/stable - DINS 序列化
├── protobuf/0.3.4@infra-cpp-cicd/stable - Protobuf
├── daf-proto-rhp/2.1.8@daf-cicd/stable - DAF Proto 运行时
├── daf-binxray/3.6.0@daf-cicd/stable - BinXray 检测/描述
├──
├── middleware/8.9.0-alpha.10@RHP/release - 中间件 (DCOS/PHM/BAG等)
├──   ├── requires: dji-message
├──   ├── requires: daf-proto-rhp, daf-binxray, daf-dins
├──   └── requires: bsp-qcomm/4.43.0-beta.2@RHP/release
├──
├── dsar-plat/0.2.0-alpha.27@RHP/release - * MCU平台包 (dsar-hq-plat编译产出)
├──   └── 提供: dsar_plat_fw 头文件、ACU设备抽象、xwire、proxy_comm等
├──
├── dsar-plat-bf/2.31.0-beta.6@RHP/release - * BF平台包 (dsar-hq-plat编译产出)
├──   ├── requires: dji-message, middleware
├──   ├── 提供: plat_bf_cdd 静态库、x_dom_can_rt/x_dom_someip_rt 源代码
├──   └── 提供: pal_tool (参数生成工具)
├──
├── dsar-plat-ad/2.1261.0-beta.87@RHP/release - * AD平台包 (dsar-hq-plat编译产出)
├──   ├── requires: dsar-plat-bf, dji-message, middleware
├──   ├── requires: adc-protocol/V0.145 - ADC协议
├──   ├── requires: visualization/3.40.10@RHP/release - 可视化
├──   ├── requires: chais/1.1.7@RHP/release - CHAIS算法库
├──   └── requires: perception-common/1.0.0
├──   └── 提供: plat_ad_app_core, plat_ad_fusa_app, plat_vehicle_proxy等
├──
├── [Git Submodules]
├──   ├── dsar-bsw (AUTOSAR BSW代码)
├──   ├── dsar-sip (SIP信号接口协议)
├──   └── ctrl/control (控制算法)
```

平台仓三个 Conan 包的版本在以下文件中管理:

- [conan_version_dsar_plat.py](#) — dsar-plat 版本
- [conan_version_dsar_plat_ad.py](#) — dsar-plat-ad 版本
- [conan_version_dsar_plat_bf.py](#) — dsar-plat-bf 版本

五、最终产物产出

从编译日志末尾可以看到:

产物	大小	内容
dsar_*_img.img	6.3M	MCU 固件镜像 (sailsw3 + sailsw1 两个变体)
dsar-app_*_img.img	69M	SOC 应用 QNX6 文件系统镜像 (含所有 .so 和可执行文件)
dsar-hq_*.tar.xz	137M	完整发布包 (含符号表 backup)

SOC 应用镜像内部大致结构:

```

/mnt/dji/apps/dsar-app/
├─ bin/
│   ├── dji_ad_app      - AD应用主进程
│   ├── dji_bf_app      - BF应用主进程
│   ├── dji_doip_service_ipv4 - DoIP诊断服务v4
│   ├── dji_doip_service_ipv6 - DoIP诊断服务v6
│   ├── dji_log_export  - 日志导出工具
│   ├── test_log_export - 日志导出测试
│   └─ tools/          - 诊断工具集
│       ├── ecu_shell, dsar_shell, dshell
│       ├── uss_cmd_test, topic_recoder, period_execute
│       └─ ...
├─ lib/
│   ├── libfawhq_e009_app.so - AD应用动态库
│   ├── libfawhq_e009_com.so - 通信动态库
│   ├── libuds_v4.so        - UDS诊断v4动态库
│   ├── libfawhq_e009_uds_v6.so - UDS诊断v6动态库
│   └─ ... (Conan依赖的 .so)
├─ etc/
│   ├── scripts/         - 运行时脚本
│   ├── runtime_param/   - 运行时参数JSON
│   ├── pal_param/       - PAL生成的参数文件
│   ├── global_var_info/ - 全局变量调试信息
│   └─ data/            - 数据文件

```

六、关键依赖链条总结

平台包 (dsar-hq-plat)

```

├─ 编译产出 → dsar-plat (MCU平台包)
│   ├── 被 dsar_fw 的 Makefile 通过 conanbuildinfo.mak 引用
│   └─ 提供: 头文件 (acu_device, xwire, proxy_comm, rtmon, can_sync等)
├─ 编译产出 → dsar-plat-bf (BF平台包)
│   ├── 被 dsar_app 的 CMake 引用 (plat_bf_cdd 静态库)
│   └─ 提供: x_dom_can_rt / x_dom_someip_rt 源代码、PAL工具
├─ 编译产出 → dsar-plat-ad (AD平台包)
│   ├── 被 dsar_app 的 CMake 引用 (plat_ad_* 静态库)
│   └─ 提供: AD应用核心、车辆信号代理、功能安全、IA平台
│       └─ 依赖: adc-proto, visualization, chais, perception-common

```

适配包 (dsar-hq)

```

├─ src/dsar_fw/ → Makefile 构建 → MCU固件 (.bin/.elf)
│   ├── 依赖: dsar-plat 头文件 + dsar-bsw submodule + control submodule
│   └─ 自有: app_core, ctrl, proxy_comm, xwire, product 等
├─ src/dsar_app/ → CMake 构建 → SOC应用 (.so + 可执行文件)
│   ├── 依赖: dsar-plat-ad + dsar-plat-bf + dsar-plat + middleware
│   └─ 自有: proxy(信号适配), adapter, param, fusa, product等
└─ Git Submodule: dsar-sip (SIP协议栈)

```

一句话总结: 编译时, Conan 先解析 14 个依赖包 (含 3 个平台包), 然后 `conan_build.py` 将源码复制 2 份 (sailsw3/sailsw1), 并行启动 Makefile (编译 MCU 固件 → 最终链接成 .bin/.elf) 和 CMake (编译 SOC 应用 → 生成 4+ 个可执行文件和 4 个 .so 动态库), 最后用 `mkqnx6fsimg` 打包成 QNX6 文件系统镜像。